Top-K Algorithm

HJ

November 7, 2026

1 Introduction

Top-K is a classic algorithmic problem, nowadays widely used in AI. This essay only discusses the algorithm itself.

2 Sequential Implementation

- **Sort**: sort the N elements and take the top K. Complexity: $O(N \log N)$.
- **Bubble**: run K bubble passes; each pass bubbles the next-largest element into place. Complexity: $O(NK)$.
- **Heap**: scan the numbers while keeping the K largest in a **min-heap** of size K. Complexity: $O(N \log K)$.
- **Quick Select**: stop when a pivot lands at the K-th largest position; everything to its right is then the K largest elements. Complexity: $O(N)$ on average, $O(N^2)$ in the worst case.

3 Parallel Implementation

Here are our two old friends, both have $O(N)$ work and $O(\log N)$ span.

- **Bucket Sort**: parallels Quick Select — apply a **parallel prefix sum** over the bucket sizes (a sequential scan is also fine, since there are usually few buckets), find the "pivot bucket" that contains the K-th element, take every higher bucket outright, and recurse only on the pivot bucket.
- **Radix Sort (MSD)**: like radix buckets — for example, take the highest 4 bits, which gives 16 buckets.

3.1 Bitonic Sort, the GOAT

A sequence is **bitonic** if it first increases and then decreases, or is a cyclic rotation of such a sequence.

Algorithm: For a bitonic sequence of length $2n$, compare&swap $x[i]$ with $x[i+n]$ so that $x[i] \le x[i+n]$; this yields 2 **bitonic** subsequences $x[0:n) \le x[n:2n)$, the bitonic property preservd by Batcher's theorem.

Work & Span: Obviously, Bitonic Sort is **extremely highly parallelizable**: every compare-and-swap can be done by a single thread. Although its total work is even worse than merge sort's, its span is **polylogarithmic**.

There is no early termination for top-K, but good enough.

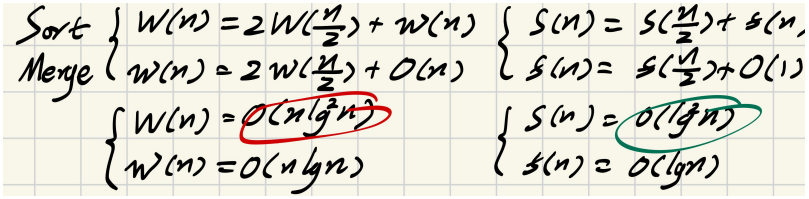


Figure 1: Bitonic Sort work and span analysis.

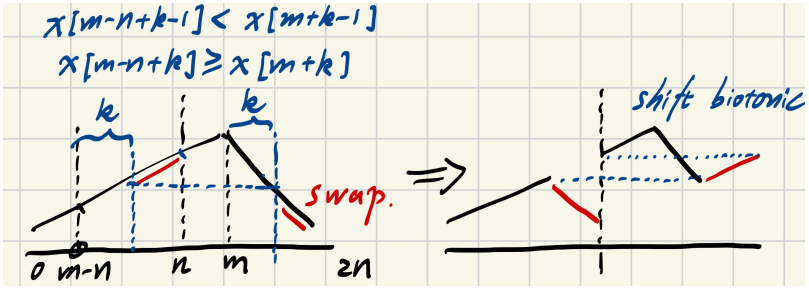


Figure 2: Batcher's theorem

TODO: Add Cuda details?